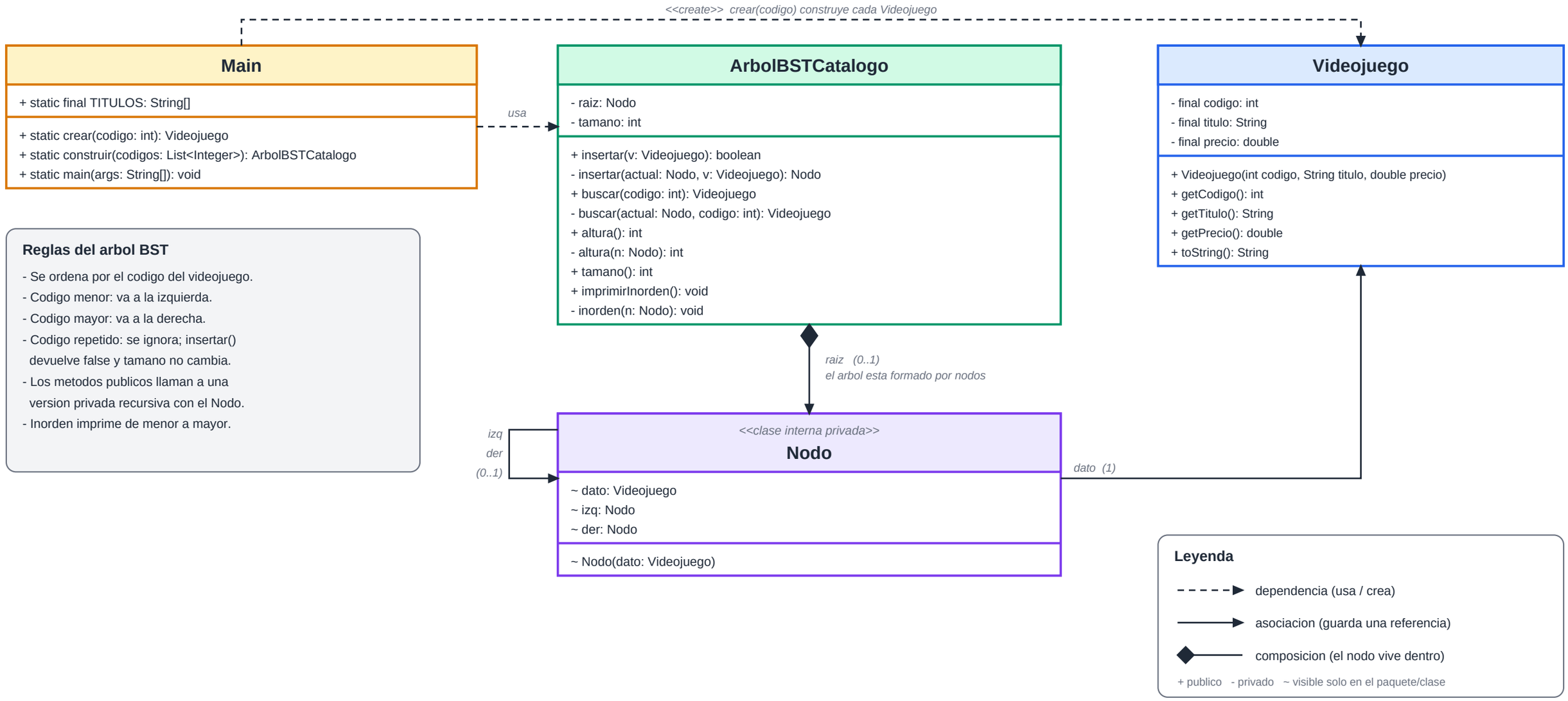
Diagrama de clases - Catalogo de videojuegos con arbol BST

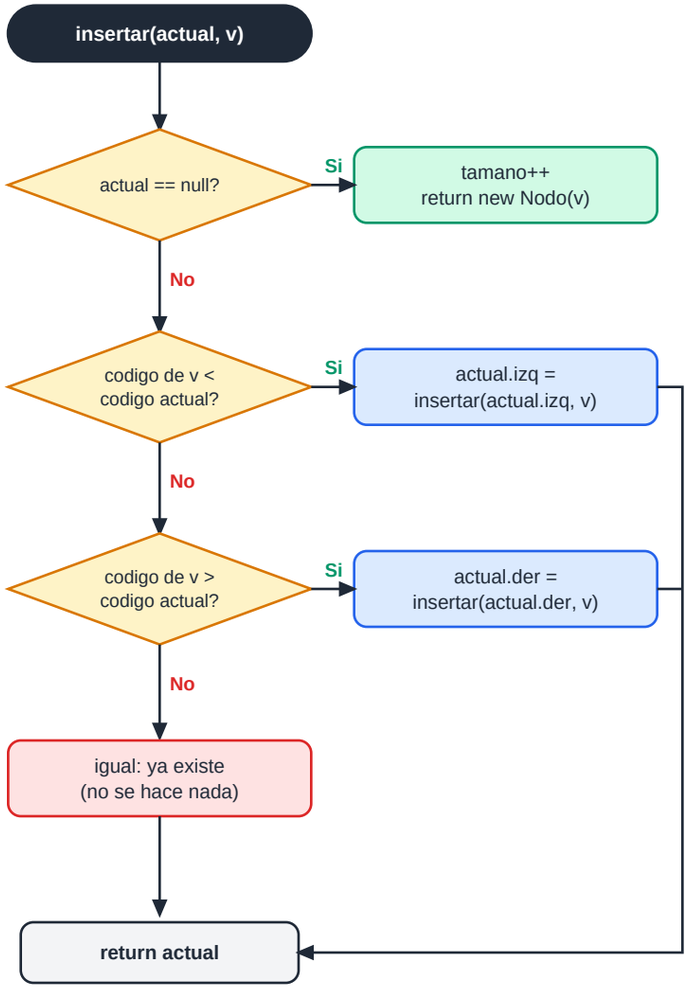
Sesion 9 | Videojuego.java - ArbolBSTCatalogo.java - Main.java



ArbolBSTCatalogo - Diagramas de flujo de los metodos recursivos

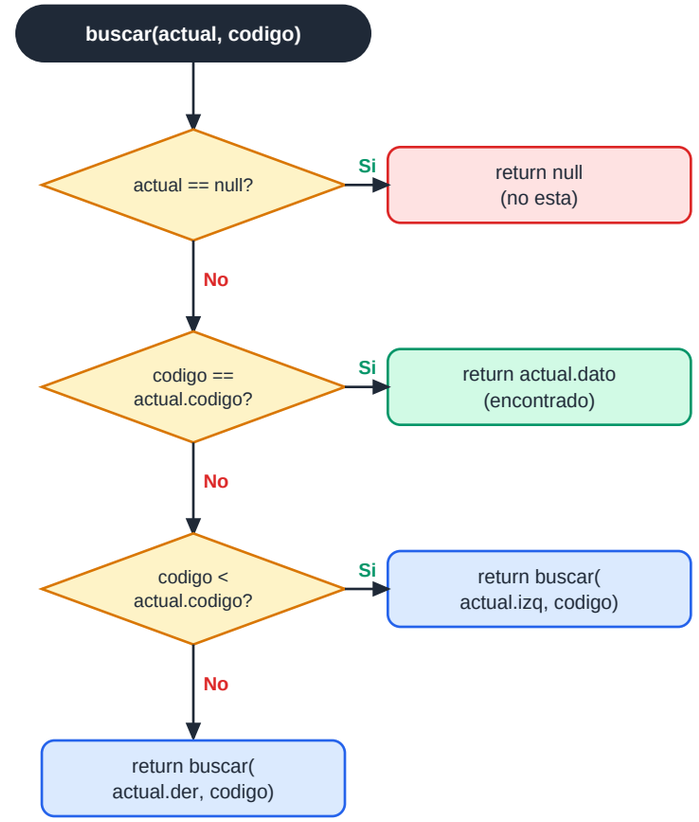
insertar - buscar - altura - inorden

insertar(actual, v)



Recursion: cada llamada baja un nivel hasta encontrar un hueco (null). Ahi nace el nodo. Al volver, cada nivel devuelve su propio nodo.

buscar(actual, codigo)

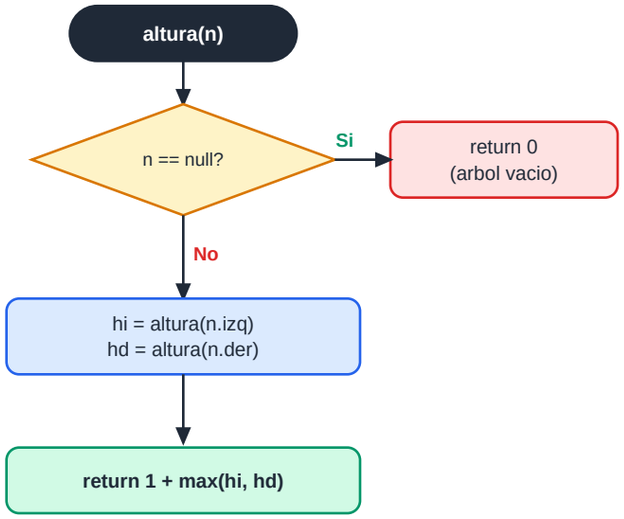


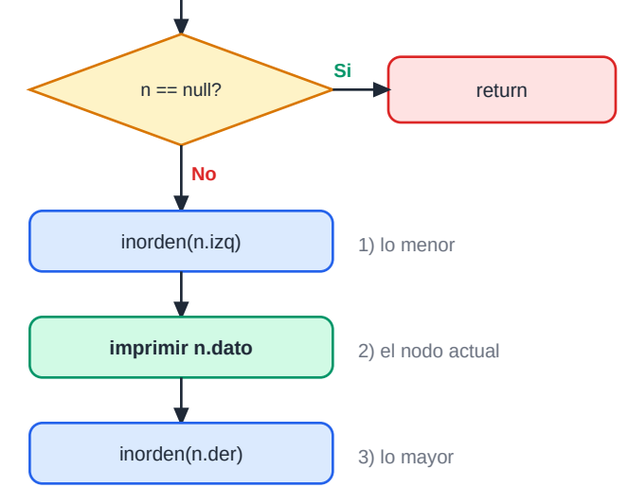
Ejemplo: buscar(60) en el arbol de la prueba

1) raiz 50: 60 es mayor, se va a la derecha
2) nodo 70: 60 es menor, se va a la izquierda
3) nodo 60: coincide, se devuelve el Videojuego

buscar(99) llega a un hueco null y devuelve null.

altura(n) e inorden(n)





Main.java - Flujo del programa y resultados

Prueba funcional + comparacion de alturas entre datos ordenados y aleatorios

Main.main(): flujo

inicio main

1. PRUEBA FUNCIONAL

demo = construir([50, 30, 70, 20, 40, 60, 80, 30])
por cada codigo: bst.insertar(crear(codigo))

tamano() = 7
el 30 repetido se ignora (insertar devuelve false)

imprimirInorden(): lista ordenada de menor a mayor codigo

buscar(60) devuelve el juego | buscar(99) devuelve null

altura() = 3

2. COMPARACION DE ALTURAS

tamanos = {10, 100, 500, 1000, 5000}
repeticiones = 10, Random(42)

para cada n de tamanos

ordenados = 1, 2, 3, ..., n
hOrd = construir(ordenados).altura()

repetir 10 veces: mezclar la lista y construir el arbol
hAle = promedio de las 10 alturas

hIdeal = ceil(log2(n + 1)) (arbol perfectamente balanceado)

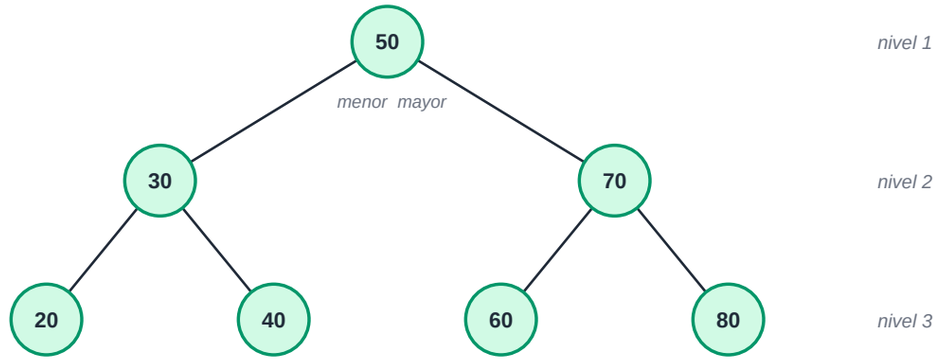
imprimir fila: n, hOrd, hAle, hIdeal, hOrd / hAle

sig. n

ya no hay mas n

fin

Arbol de la prueba funcional (altura 3)

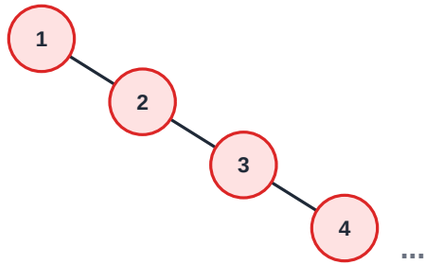


inorden: 20, 30, 40, 50, 60, 70, 80 (sale ordenado)

buscar(60): 50, derecha a 70, izquierda a 60

Para cada juego: titulo = TITULOS[codigo % 10] + " #" + codigo; precio = 10 + codigo % 60

Datos ordenados: el arbol se vuelve una lista



Insertar 1, 2, 3, ... n:

cada codigo es mayor que el anterior, siempre se va a la derecha.

altura = n (cada insercion recorre toda la rama) y buscar cuesta O(n).

Con orden aleatorio la altura crece de forma parecida a log2(n).

Resultados (salida.txt)

n	Altura ordenado	Altura aleatorio*	Altura ideal	Ord/Aleat
10	10	5.2	4	1.9x
100	100	13.2	7	7.6x
500	500	19.2	9	26.0x
1000	1000	21.4	10	46.7x
5000	5000	28.5	13	175.4x

** promedio de 10 corridas con orden aleatorio*

Conclusion: el BST sin balancear solo es eficiente si los datos llegan en desorden.